

AI-I04 | 现代 LLM 应用系统

Context、Retrieval、RAG、Memory、Agent、Workflow 与 MCP

技术快照：2026-08-22

[返回 Integration Landing Page](#)

Contents

1 本册定位：从语言模型到可运行系统	4
1.1 本册与各 Core Area 的 Ownership	5
2 Context：模型这一轮真正看见什么	6
3 Retrieval：从外部知识中选择当前证据	7
3.1 Embedding 是表示，不是知识库本身	7
3.2 Chunking：检索单元必须服从文档结构	8
3.3 Retrieval 是一条筛选链，不只是最近邻搜索	8
4 RAG：把外部证据送入当前生成过程	9
4.1 RAG 与 Fine-tuning 的边界	10
4.2 Grounded Generation 仍然可能失败	10
5 Model API：当前应用层怎样调用模型	11
5.1 模型选择是一项系统决策	11
6 Tool Calling：模型提出动作，Runtime 负责执行	11
7 Agent：核心是受控循环	12
7.1 Planning 与 Stopping 必须显式设计	13
8 Agent Harness：把一个模型变成可长期运行的代理系统	14
8.1 长时程 Agent 的关键分离：Session 不等于 Context Window	15
8.2 Brain-Hands-Session：长时程 Agent 的故障域要解耦	16
8.3 Harness Scaffolding 是关于“模型目前做不到什么”的可检验假设	17
9 Skill、Tool、MCP 与 Subagent：四种接口不要混成“Agent 能力”	17
9.1 Defaults、Examples 与 Feedback：把领域经验变成可维护行为	18
10 State 与 Memory：先区分“当前执行”与“以后还能记得”	19
10.1 Long-term Memory 的母模型：写入、维护、检索、注入是四个决策点	19
10.2 第一维：记什么——Semantic、Episodic、Procedural	20
10.3 第二维：怎么存——五种常见架构模式可以叠加	21
10.4 第三维：怎么找——Preload、Lexical、Semantic、Graph/Hybrid	22
10.5 第四维：怎么维护——不要把“新事实”简单等同于覆盖旧行	22
10.6 Hot Path 与 Background：记忆形成发生在什么时候	23

10.7 当前实现差异：产品名不能替代架构判断	23
10.8 Memory 的安全边界：持久化错误比一次 hallucination 更危险	24
10.9 Memory 怎么评：不能只问“最终回答看起来对不对”	25
11 Loop 与 Graph：开放探索和显式流程不是二选一	25
12 Multi-Agent：只有任务真的可分解时，并行委派才有价值	26
12.1 什么时候值得使用多个 Agent	27
12.2 Deep Research：一个可复用的 Integration 母例	27
12.3 Planner-Generator-Evaluator：多 Agent 的另一条价值是分离“做”和“判”	28
13 LangChain 与 LangGraph：当前分工	28
13.1 LangChain v1：高层 Agent API	28
13.2 Deep Agents：当前一个明确的 Harness 实现	29
13.3 LangGraph v1：显式 State + Node + Edge	29
14 MCP：标准化外部 Context 与 Capability 接口	30
14.1 当前 MCP 基线：2026-07-28	31
14.2 MCP Python SDK v2	31
14.3 Tool Schema 与错误边界	32
15 Security：任何 RAG / Tool / MCP 系统都要画 Trust Boundary	32
15.1 先限制 Blast Radius，再讨论模型会不会犯错	33
16 Observability、Evaluation、Cost 与 Failure Recovery	34
16.1 Observability：必须能重建一次执行轨迹	34
16.2 Evaluation：按层拆开，而不是只看“回答准确率”	34
16.3 Agent Eval 的基本单位：Task、Trial、Grader、Transcript	35
16.4 Agentic Benchmark 里 Runtime 本身就是实验变量	35
16.5 LLMOps：Trace 只有进入评测与回放闭环才会产生改进	36
16.6 Cost 与 Latency 是多步系统的累积量	36
16.7 Failure Recovery：失败是运行时的一部分	37
17 Complexity Ladder：什么时候不要继续增加 Agent 自主性	38
18 一条可执行的实验路线	38
18.1 Lab A 直接 Model API	38
18.2 Lab B Semantic Retrieval	39

18.3 Lab C RAG	39
18.4 Lab D Function / Tool Calling	39
18.5 Lab E LangChain Agent	39
18.6 Lab F LangGraph Workflow	39
18.7 Lab G MCP Server	39
18.8 Lab H Production Evaluation	39
18.9 Lab I Long-term Memory Lifecycle	40
18.10Lab J Skill、Tool 与 Harness Boundary	40
18.11Lab K Loop、Graph 与 Hybrid	40
18.12Lab L Orchestrator–Worker Deep Research	40
18.13Lab M Session、Context 与 Crash Recovery	40
18.14Lab N Agent Eval 的重复试验与 Runtime 控制	41
19 最终压缩：十四个问题复原整套系统	41
20 当前技术快照与官方资料	42

1 本册定位：从语言模型到可运行系统

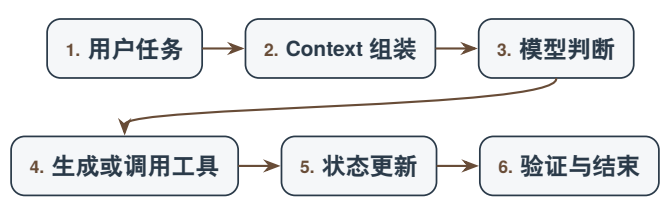
一个语言模型最基本的运行形式是“输入当前上下文，生成输出”。真实应用却还要处理私有知识、实时业务数据、工具调用、多步状态、权限、失败恢复和质量评估。

本册不重新解释 Transformer、概率、优化或搜索等单点机制，只追踪这些成熟模块怎样在一次真实任务里交接。母问题是：

Mother Question | 一次 LLM 任务到底怎样跑完

一次真实 LLM 任务执行时，哪些信息进入当前上下文，哪些知识需要检索，什么时候只生成文本，什么时候调用工具，谁控制多步流程，状态保存在哪里，外部系统怎样安全接入，最后又怎样判断这次执行是否真的成功？

最小运行链可以写成：



这条链只表示一次任务的运行顺序，不表示各阶段在理论上属于同一个机制。

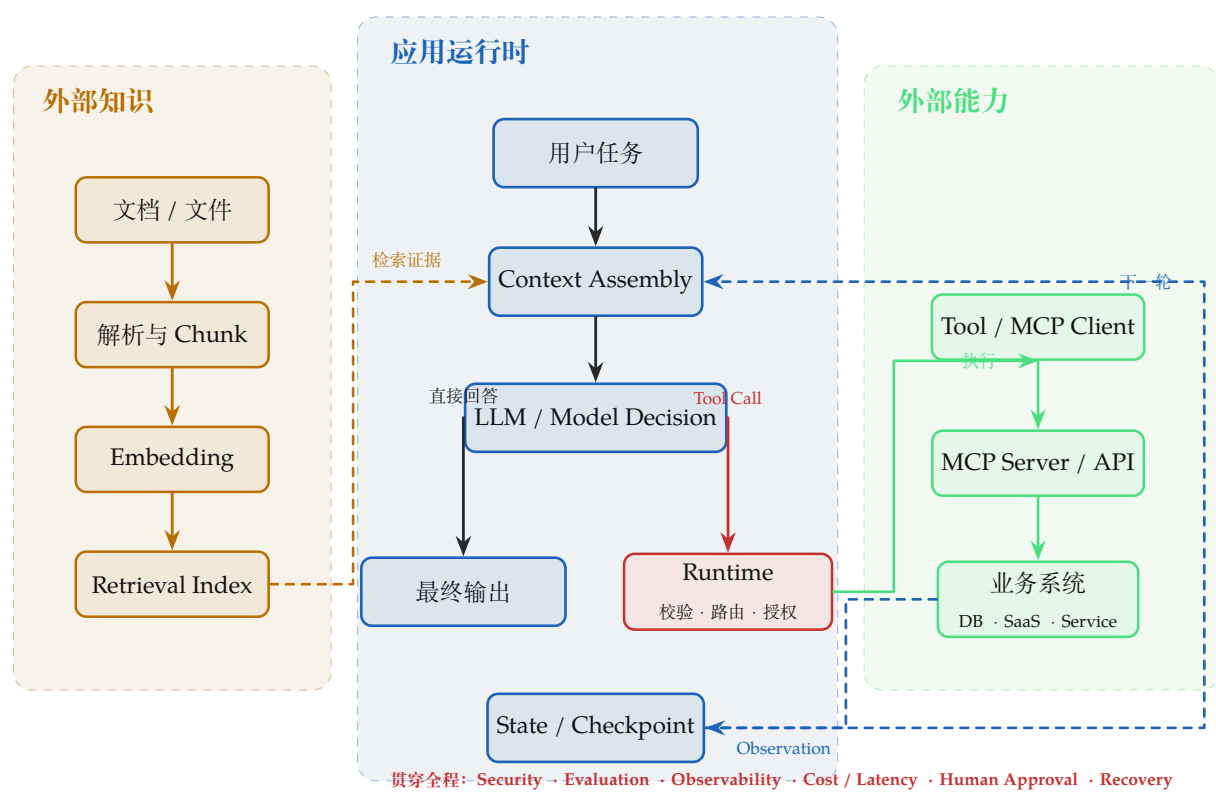


Figure 1 现代 LLM 应用系统总架构。主路径表示一次任务怎样推进；左侧是外部知识进入 Context 的检索路径，右侧是模型通过 Tool / MCP 访问外部能力的执行路径。

1.1 本册与各 Core Area 的 Ownership

对象 / 机制	Canonical Owner	AI-I04 只负责什么
Transformer / Attention	Area 60 深度学习	模型在完整系统中承担生成与推理运行时
Autoregressive generation	Area 80 生成模型	生成结果怎样接入应用流程
Learning / evaluation contract	Area 40 机器学习	系统级评测怎样拆成 retrieval、answer、trajectory
Optimization / AutoDiff	Area 50 优化与学习计算	本册只在成本或调用边界处引用
Search / decoding search	Area 10 问题求解	Agent 或生成流程何时调用搜索机制
Decision / sequential control	Area 70 决策、控制与 RL	多步行动和策略语义的接口
Context / Retrieval / RAG / Tool / Workflow / MCP 的组合	本 Integration	一次任务中对象怎样传递、状态怎样变化、控制权在哪里

必须稳定区分：

概念 A	≠	概念 B	真正区别与题目信号	混淆后果
Vector Store	≠	Long-term Memory	Vector Store 是检索索引；Memory 是跨轮次或跨任务持久保存的状态/事实	把“能检索”误当成“模型自动记住”
Retrieval	≠	RAG	Retrieval 只负责找证据；RAG 还要把证据组织进 Context 并生成回答	只评向量搜索却声称整个 RAG 有效
Tool Calling	≠	Agent	Tool Calling 是一次结构化动作接口；Agent 还需要循环、状态、停止条件和控制策略	把会调函数的单次调用误称为完整 Agent
MCP	≠	Agent	MCP 标准化外部 Context / Capability 接口；Agent 决定任务怎样推进	把协议本身当成“智能”或“安全机制”
Graph State	≠	Long-term Memory	State 服务当前 workflow；Long-term Memory 跨 thread / session 持久存在	状态越堆越大，生命周期和权限不清

2 Context：模型这一轮真正看见什么

上下文窗口（context window）表示一次模型调用能够处理的 token 工作集。它可以包含系统/开发者指令、用户输入、对话历史、检索到的文档、工具定义、工具结果、文件和任务状态。

但下面这些对象不是同一个东西：

- **预训练知识**：模型参数中已经学到的统计规律；
- **当前 Context**：这一轮请求实际送给模型的工作集；
- **Retrieval Index**：外部知识的可检索索引；
- **Application Database**：业务系统的事实与事务状态；
- **Long-term Memory**：应用显式保存、以后任务可以再次读取的信息。

截至 2026-08-22，OpenAI GPT-5.6 Sol / Terra / Luna 的 context window 都是约 1.05M tokens，最大输出为 128K tokens。长上下文扩大了“能放多少”的上限，但没有自动解决“该放什么”。

Context Engineering | 真正优化的是相关而充分的工作集

Context 越长并不天然越好。无关历史、重复资料和不可信外部文本都会增加干扰、成本和延迟。系统要做的是从“所有可用信息”中选择“当前任务需要的信息”，再把它按清楚的角色和边界组织给模型。

每轮调用至少要回答：

1. 哪些 instructions 必须保留，优先级怎样；
2. 哪些历史消息仍然相关；
3. 哪些知识应该实时 Retrieval，而不是长期堆在对话里；
4. 哪些 Tool schemas 需要暴露给模型；
5. Tool result 哪部分要进入下一轮；
6. 哪些状态应该结构化保存，而不是继续扩充消息历史；
7. 哪些旧信息已经过期；
8. 哪些外部内容必须按不可信数据处理。

因此 Prompt Engineering 是 Context Engineering 的一部分，但不是全部。

3 Retrieval：从外部知识中选择当前证据

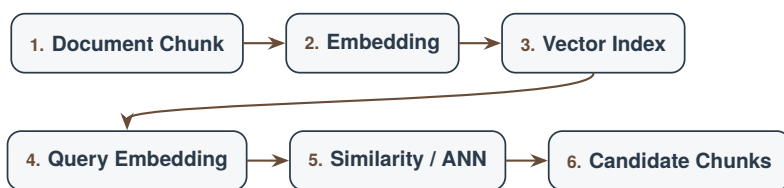
3.1 Embedding 是表示，不是知识库本身

文本 Embedding 可以写成：

$$f_{\text{embed}}(x) = v \in \mathbb{R}^d.$$

维度 d 由具体 embedding model 决定，不存在统一固定值。Embedding 的作用是把文本或其他对象映射到适合相似度计算、聚类 and 检索的 learned representation。

典型语义检索过程是：



实际相似度可以使用 cosine、dot product、Euclidean distance 等，具体取决于 embedding model 和索引设计。

截至当前版本，OpenAI 仍提供 text-embedding-3-small 与 text-embedding-3-large。text-embedding-3 系列支持通过 dimensions 调整输出维度。轻量本地教学场景中，也常见 sentence-transformers/all-MiniLM-L6-v2，它输出 384 维向量。具体型号只是实现选择，不能反过来定义 Embedding 的机制。

3.2 Chunking：检索单元必须服从文档结构

长文档通常不适合作为一个 retrieval unit。Chunk 太大时，一个向量可能混入多个主题；Chunk 太小时，定义、条件或论证链可能被切断。

Overlap 可以保护边界附近的上下文，但不存在跨数据集通用的“最佳 chunk size / overlap”。更稳的判断顺序是：

1. 先看文档原生结构；
2. 再决定一个可独立理解的语义单元；
3. 再看真实 query 通常需要多大证据范围；
4. 最后结合 Context budget 与 retrieval eval 调参数。

例如：

- 法律文本优先保留条、款、定义和引用关系；
- API 文档尽量保持函数、参数、返回值完整；
- 对话记录可以细分，但要保留说话者和必要邻近轮次；
- 教材不应机械切断定义、定理、证明和例题的局部闭环。

OpenAI 当前 Vector Store 的 auto chunking 默认值为：

```
1 max_chunk_size_tokens = 800
2 chunk_overlap_tokens = 400
```

这是当前产品默认值，不是理论最优值。生产系统仍然要用自己的查询集做 retrieval eval。

3.3 Retrieval 是一条筛选链，不只是最近邻搜索

完整 retrieval pipeline 通常包含：



当前 OpenAI Vector Store Search 已显式提供 attribute filters、max_num_results、ranker、score_threshold 和 query rewrite 等能力。这说明检索真正要解决的是“候选怎么生成、怎样过滤、怎样排序、最终哪些证据进入 Context”。

若接受阈值为 τ ，提高 τ 往往会提高 **precision**，同时可能降低 **recall**。阈值必须由任务风险和测试集决定，不能只凭感觉指定。

SQL 与 Vector Retrieval 也不是二选一。同一个系统可以同时使用结构化过滤、全文搜索、向量相似度、权限过滤与 reranker；例如 PostgreSQL 配合 pgvector 就能在一个数据系统中完成结构化查询和向量近邻搜索。

先判断问题需要哪一种“真相接口”

不要因为系统里有 Vector Store，就把所有数据都改写成 Embedding。先看用户问题要求什么类型的答案：

问题对象	优先接口	为什么
精确结构化事实	SQL / typed API / RPC	ID、价格、库存、状态、时间范围、聚合和权限过滤需要确定语义与精确条件
长文本中的语义证据	FTS / Vector / Hybrid Retrieval	用户表述与原文未必共享关键词，需要按语义召回相关段落
实时外部事实	Tool / API	当前值属于外部系统事实，应在请求时读取，而不是依赖旧 Context 或旧 embedding
有副作用的动作	Authorized Tool / Workflow	发邮件、退款、写数据库等必须经过 schema、权限、幂等和审计边界

一个 Agentic RAG 系统真正“agentic”的地方，不是 Vector Store 本身，而是 Runtime 根据当前意图和证据需求，在这些接口之间做路由。例如“融资政策是什么”可能需要从政策文档做语义检索；“当前有哪些蓝色且低于 3 万美元的库存”应优先查询结构化库存表；“把其中一辆加入试驾日程”才进入有副作用的 Tool / Workflow。三类问题可以出现在同一次对话中，但不能共用同一种数据访问语义。

4 RAG：把外部证据送入当前生成过程

RAG (Retrieval-Augmented Generation) 的责任链是：

- 1. **Retrieval**：当前问题需要哪些外部证据；
- 2. **Augmentation**：怎样把证据、来源和用户问题组织进当前 Context；
- 3. **Generation**：模型怎样基于这组证据生成回答。

因此：

RAG ≠ Vector Search

Vector Search 只是 Retrieval 的一种实现。RAG 还拥有 request-time context construction 与 generation 这两段组合责任。

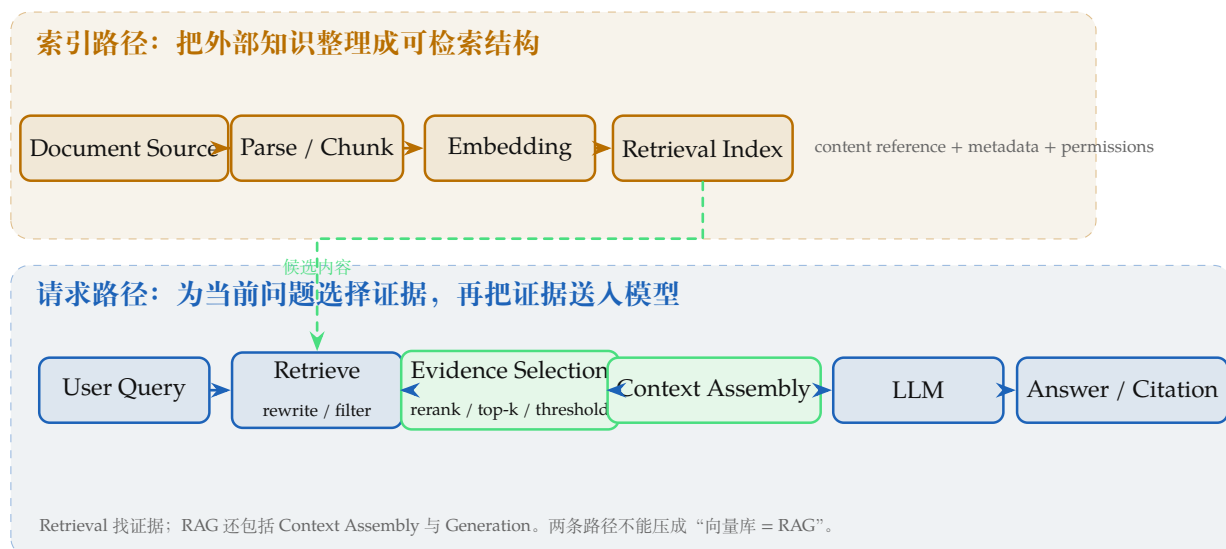


Figure 2 RAG 的两条路径。上半部分是离线/增量索引路径，下半部分是请求时路径；二者在 Retrieval Index 处汇合，检索出的证据经过 Context Assembly 才进入模型。

4.1 RAG 与 Fine-tuning 的边界

Fine-tuning 改变模型参数或行为分布；RAG 不修改基础模型权重，而是在 inference time 提供外部证据。频繁更新的企业知识、需要 citation 的问答、私有文档通常优先考虑 Retrieval / RAG；行为风格、稳定格式和高频任务模式则可能更适合 fine-tuning。两者也可以组合。

4.2 Grounded Generation 仍然可能失败

即使系统要求“只根据提供的文档回答”，仍然可能出现：

- Retrieval miss：正确证据没有进入 Context；
- Retrieval false positive：检索了看似相似但实际不相关的内容；
- Stale source：证据已经过期；
- Conflicting sources：多个来源互相冲突；
- Incorrect synthesis：证据正确，但模型组合或解释错误；
- Citation mismatch：回答与引用证据并不真正对应。

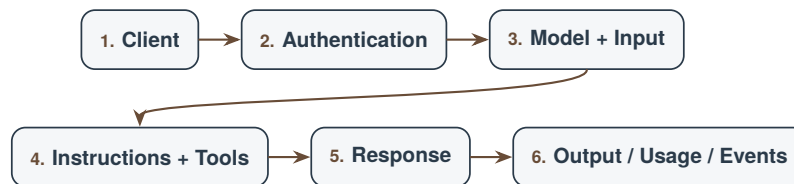
因此 RAG Evaluation 至少要拆成 Retrieval Quality 与 Generation / Grounding Quality 两层，不能只用一个“回答准确率”覆盖全部失败原因。

5 Model API：当前应用层怎样调用模型

截至 2026-08-22，OpenAI 当前文档建议使用 Responses API 构建 reasoning、tool-calling 与 multi-turn workflow。最小 Python 调用可以写成：

```
1 from openai import OpenAI
2
3 client = OpenAI()
4 response = client.responses.create(
5     model="gpt-5.6",
6     input="Explain retrieval-augmented generation in one paragraph.",
7 )
8 print(response.output_text)
```

真正应该记的是调用契约，而不是某一版 response object 的路径：



Responses API 当前可以组合 function tools、web search、file search、code interpreter、computer use、remote MCP 等能力。模型调用已经不再只是“一问一答”，而可以成为受控工具运行时的一部分。

5.1 模型选择是一项系统决策

GPT-5.6 family 当前包括：

- gpt-5.6-sol / gpt-5.6：能力优先；
- gpt-5.6-terra：能力与成本平衡；
- gpt-5.6-luna：高吞吐、成本敏感。

三者当前 context window 均约 1.05M tokens。选择模型时不能只看公开 benchmark，还要在自己的任务上比较 Task Success、Quality、Latency、Token Usage、Tool-call Accuracy 与 Cost。

6 Tool Calling：模型提出动作，Runtime 负责执行

Tool 至少应明确：

- name 与 description；

- input schema;
- output contract;
- permission boundary;
- side-effect semantics;
- failure contract。

一次 Tool Call 的责任链是：



所以“模型调用工具”并不意味着模型自己执行了外部副作用。真正执行数据库写入、发邮件、调用业务 API 的仍然是应用 Runtime 或 Tool Server。

7 Agent：核心是受控循环

Agent 不是“会用工具的更聪明 LLM”。它至少包含模型决策、工具系统、状态、循环与停止条件。

一次任务可以还原为：

1. 接收用户任务；
2. 组装当前 Context；
3. 模型选择回答、Tool Call、route / handoff 或 stop；
4. Runtime 校验动作；
5. Tool 在权限、timeout、retry 和副作用规则下执行；
6. Observation 返回；
7. 更新 conversation / task state / checkpoint / trace；
8. 继续下一轮或终止；
9. 高风险动作必要时插入人工批准；
10. 全程记录 token、latency、cost 与 error。

主循环只回答“下一步怎样执行”；人工批准和 checkpoint 都是旁路控制，不能画成模型自主性的组成部分。

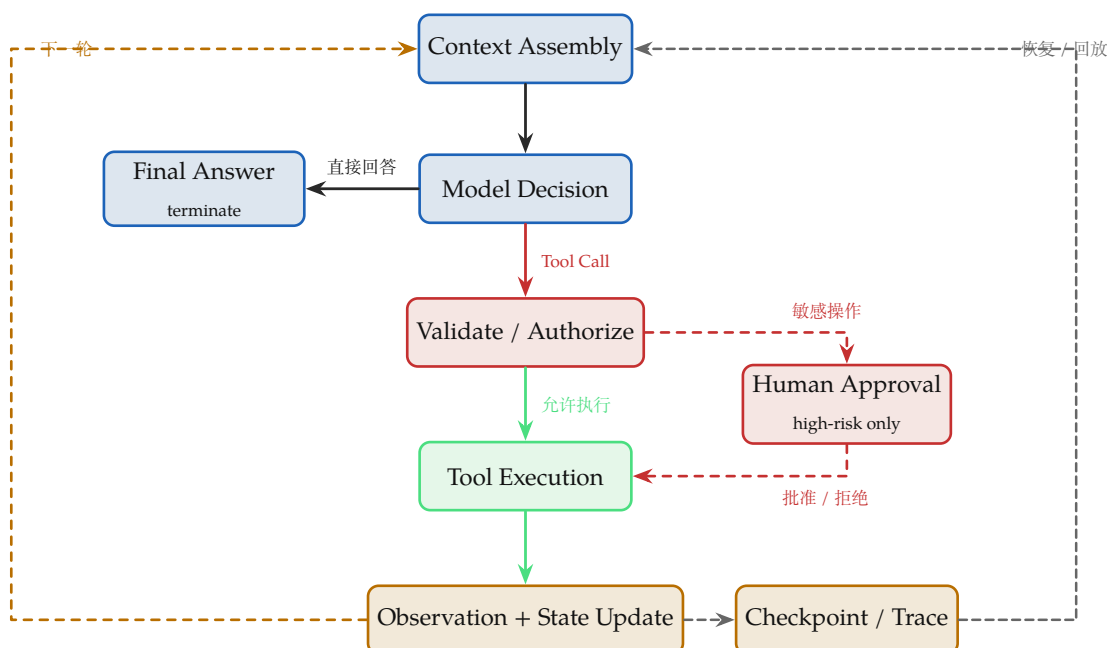


Figure 3 Agent 受控运行时循环。模型负责提出下一步候选动作，Runtime 负责验证与执行；高风险操作可以在执行前进入人工批准，Observation 和 State 再回到下一轮。

7.1 Planning 与 Stopping 必须显式设计

一个真实 Agent 至少要回答：

- 谁决定下一步；
- 最多运行多少轮；
- 什么条件算任务完成；
- 什么时候因为证据不足停止；
- 什么时候请求用户确认；
- 什么时候转给人工；
- 怎样避免循环和重复副作用。

Agent 也不必什么都负责。Routing、Planning、Retrieval、Tool Selection、Execution、Verification、Summarization 与 Escalation 可以分别由模型、确定性代码、workflow edge 或人工拥有。

8 Agent Harness：把一个模型变成可长期运行的代理系统

“Agent Harness”不是模型能力的同义词，也没有一个跨所有厂商完全统一的字段清单。这里采用当前工程实践中最有操作性的定义：**Harness** 是围绕模型建立的一组运行时约束与支撑机制，使模型能够在明确的上下文、工具、状态、权限、循环和观测边界内持续完成任务。

可以把它压成：

Agent Harness = Context + Tools + Loop / Planning + Session / State + Memory / Artifacts + Policy / Permissions

模型仍然负责根据当前输入产生候选文本或动作；Harness 决定模型这一轮看到什么、能调用什么、动作能否执行、怎样继续下一轮、状态怎样持久化以及失败后怎样恢复。换一个更强的模型，并不会自动替你完成这些系统责任。

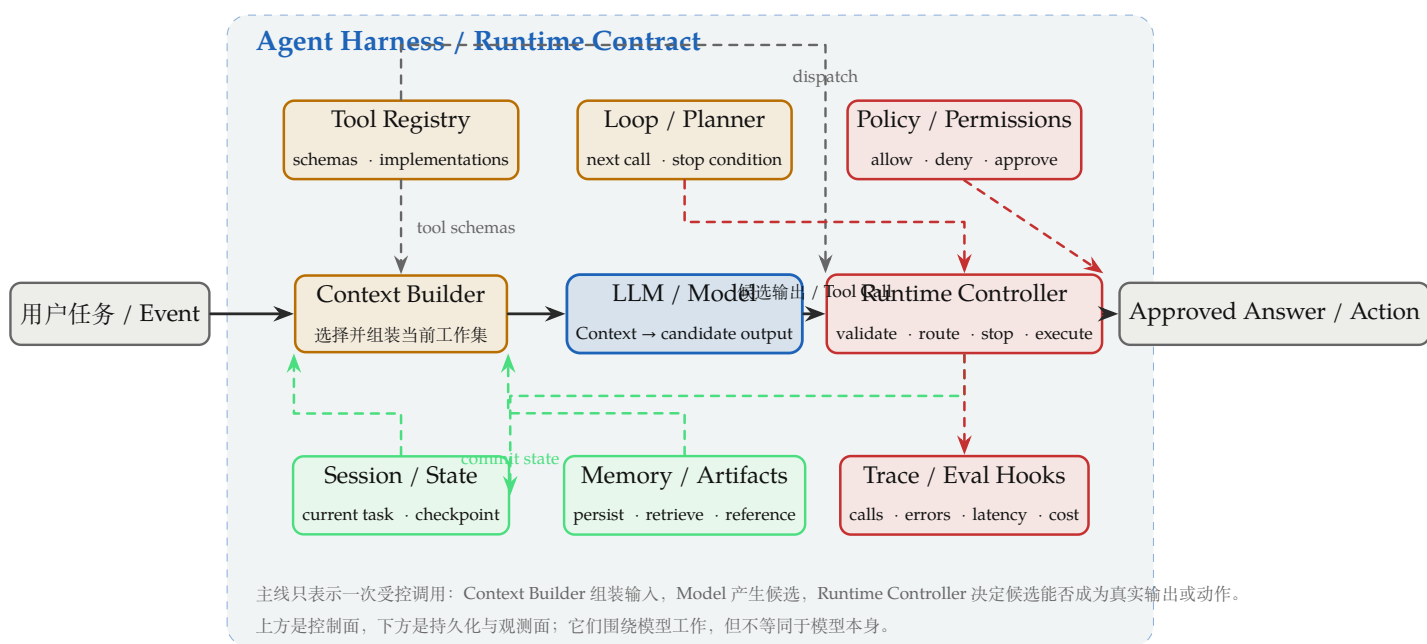


Figure 4 Model 与 Agent Harness 的责任边界。主调用链是“Context Builder → Model → Runtime Controller”；上方控制面提供 Tool Registry、Loop/Planner 与 Policy，下方持久化面提供 Session/State、Memory/Artifacts 与 Trace/Eval。模型只产生候选输出，真实动作必须经过 Runtime Controller。

一个 Harness 是否值得加入某个组件，要由真实 Failure 决定。例如：

- 长任务跨 Context 丢失进度 ⇒ 外部 artifact / checkpoint / compaction；

- 工具输出淹没主上下文 $\Rightarrow$ filesystem offload 或 subagent context isolation;
- 模型过早宣布完成 $\Rightarrow$ explicit completion criteria + verification;
- 写操作重复执行 $\Rightarrow$ idempotency key / checkpoint / side-effect ledger;
- 高风险动作越权 $\Rightarrow$ permission policy + human approval;
- 出错后无法解释 $\Rightarrow$ trace + structured state transitions。

因此 Harness 不是“功能越多越高级”。模型能力、任务结构和运行环境改变后，原来必要的 planning、memory、subagent 或 prompt scaffolding 可能变成多余成本。维护 Harness 时要持续重新验证每个组件是否仍然承担真实责任。

8.1 长时程 Agent 的关键分离：Session 不等于 Context Window

短任务里，conversation history、当前 Context 与“这次任务发生过什么”看起来几乎是一回事；一旦任务跨越多个 context windows、进程重启或多小时执行，这三个对象就必须拆开。

更稳定的模型是：

Session = 可恢复的事件历史与任务事实源

而第 t 轮真正送入模型的只是它的一张工作视图：

$Context_t = \text{Assemble}(\text{Select}(\text{Session}, \text{State}, \text{Artifacts}), \text{Memory}, \text{Retrieved Evidence})$

这里 Select 可以包含 slicing、trimming、compaction、summary、prompt-cache-friendly organization 等策略。因此：

- **Session 要可恢复**：原始 events、tool results、状态变化和关键 artifacts 不应因为当前 Context 被压缩就永久消失；
- **Context 可以有损**：为了 token budget，可以只选择当前步骤真正需要的局部历史；
- **Compaction 不是持久化**：summary 只是新的 Context 表示，不能替代原始可审计历史；
- **Checkpoint 也不是完整 Session**：checkpoint 负责恢复当前执行状态，session log 负责保留更完整的事件轨迹。

Anthropic 在 2026 Managed Agents 的工程设计中把这层边界明确成 append-only session log：Harness 可以按位置重新读取历史 events，再选择怎样组织成新的 Context。这个实现不是唯一标准，但它揭示了一个可迁移原则：**可恢复历史应活在 Context Window 外部，Context Engineering 只负责构造当前工作集。**

8.2 Brain-Hands-Session：长时程 Agent 的故障域要解耦

把 Harness、执行环境和会话历史全部放进同一个容器，短期实现很直接，但会把三个本来不同的 failure domain 绑在一起。更稳的接口化结构是：

- **Brain**：Model + Harness，负责读 Context、做决策、路由 Tool、处理 Observation；
- **Hands**：Sandbox / Workspace / MCP / External Tools，负责真实文件、代码、网络和业务副作用；
- **Session**：独立持久的事件日志与任务状态，用于恢复、回放和重新构造 Context。

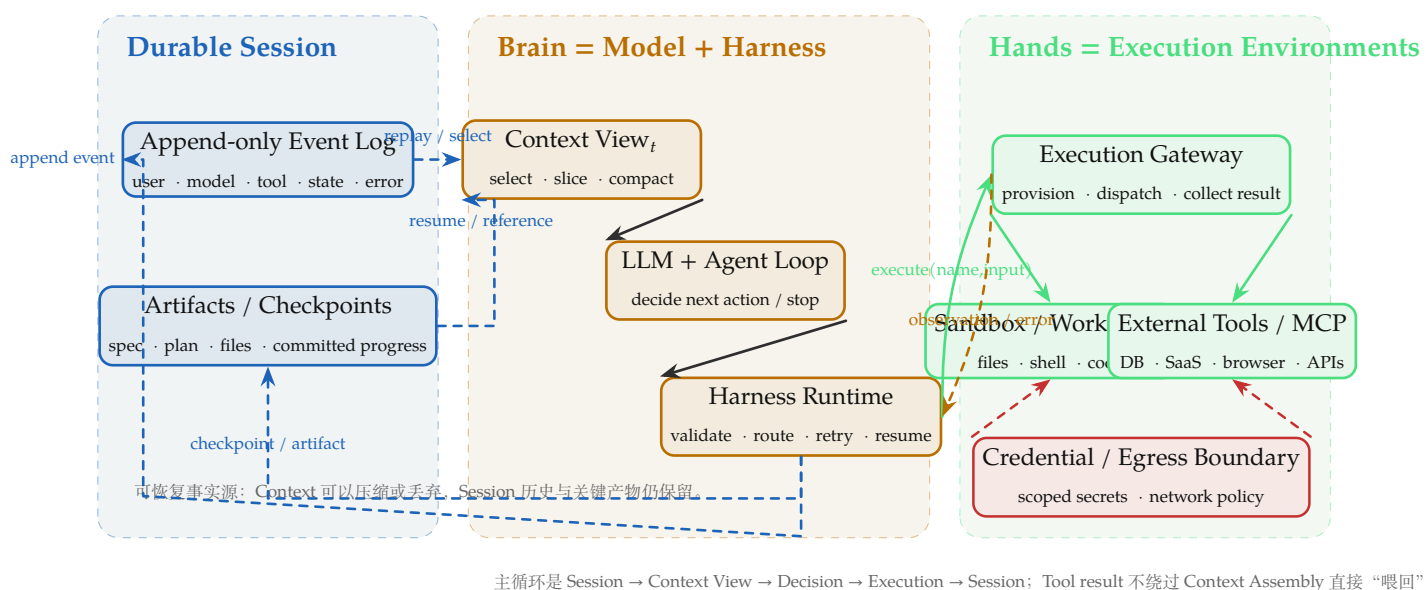


Figure 5 长时程 Agent 的 Brain-Hands-Session 解耦。Session 保存可恢复历史，Harness 从中选择当前 Context；Sandbox / Tools 作为可替换执行环境承接副作用。这样 Harness crash、sandbox crash 与 context compaction 不再等价于任务历史丢失。

这一分离带来三个重要结果：

1. **Recoverability**：Harness 进程可以无状态重启，再从 Session 恢复；Sandbox 失败可以作为一次 Tool Error 返回，并重新 provision；
2. **Security**：credentials 可以留在 sandbox 外部的 vault / proxy，由 Runtime 代理调用，而不是暴露给 Agent 生成的任意代码；
3. **Scalability**：一个 Brain 可以按需连接多个 Hands，执行环境也不必在任务开始前全部预热。

这里最值得记住的不是某个 hosted Agent API，而是接口原则：

durable state | decision process | side-effect environment

这里的竖线只表示生命周期与 **failure domain** 应显式分离，不表示统计独立或数学正交。三者可以合作，但不应因为实现方便就被迫共享同一个生命周期。

8.3 Harness Scaffolding 是关于“模型目前做不到什么”的可检验假设

Harness 中的每个 workaround 都隐含一个模型假设。例如“模型接近 context limit 会过早收尾”可能促使系统定期 context reset；“模型不会可靠检查自己的实现”可能促使系统加入独立 evaluator；“模型不知道如何拆任务”可能促使系统强制 planner / todo。

这些假设不能永久冻结。Anthropic 的长任务实验提供了一个很好的例子：早期对 Sonnet 4.5 有效的 context reset，在后续更强模型上变成额外的 orchestration / latency 开销，因此被移除并改由自动 compaction 支撑连续 session。这个结果不能推广成“以后都不需要 reset”，但它说明：

Harness 组件的保留条件

每次更换 Model、Tooling、Context Window、Sandbox 或任务分布后，都应该在固定 eval set 上重新做 ablation：移除某个 Harness 组件后，Task Success、Safety、Cost、Latency 与 Recovery 是否真的变差？若没有，就应考虑删除这层 scaffolding。

因此现代 Agent Engineering 不是不断向 Harness 加功能，而是维持一组**最小、可解释、能被失败证据证明有必要的运行时约束**。

9 Skill、Tool、MCP 与 Subagent：四种接口不要混成“Agent 能力”

它们经常一起出现，但回答的是不同问题：

对象	核心问题	稳定边界
Skill	这类任务 应该怎样做 ?	可复用的程序性知识、说明、模板、脚本与资源；本身不等于一次外部副作用
Tool	当前这一步 可以执行什么动作 ?	带 schema / contract 的可调用能力；真正副作用由 Runtime / Tool implementation 执行
MCP	外部能力与上下文 怎样以标准协议暴露 ?	标准化 Tools / Resources / Prompts 等协议接口；不决定任务规划、授权策略或是否应该调用
Subagent	哪个子任务值得 隔离到另一个执行上下文 ?	一个被委派明确目标的独立 agent run，可有自己的 prompt、tools、context 和 budget；不要求一定并行

把 Skill 类比为 procedural memory 有教学价值，因为它保存“以后遇到这类任务怎样行动”的稳

定知识；但二者不能直接画等号。Skill 通常是显式、可版本控制的工程资产，还可以打包脚本与参考资料；“procedural memory”则是更宽的认知/系统组织概念。

以当前 Anthropic Agent Skills 为代表实现，一个 Skill 目录以 SKILL.md 为入口：name / description 等 metadata 先帮助 Agent 判断相关性，正文在任务匹配时再读取，更深的 reference / script / resource 继续按需加载。这种 progressive disclosure 的价值不是形式漂亮，而是避免把所有专业知识永久塞进 Context。

Tool 与 Skill 也不能互换。一个“部署 FastAPI 服务”的 Skill 可以告诉 Agent 应检查哪些文件、依次执行哪些步骤、怎样验证结果；真正的 bash、GitHub、Cloud API 或文件写入仍然是 Tools。Skill 教流程，Tool 提供动作。

Subagent 则解决另一类问题：当一个子任务会产生大量中间搜索、需要专门工具/指令，或应与主任务隔离 Context 时，把它委派给独立 worker，主 Agent 只接收压缩结果或 artifact reference。简单单步任务不应为了“多智能体”而创建 subagent。

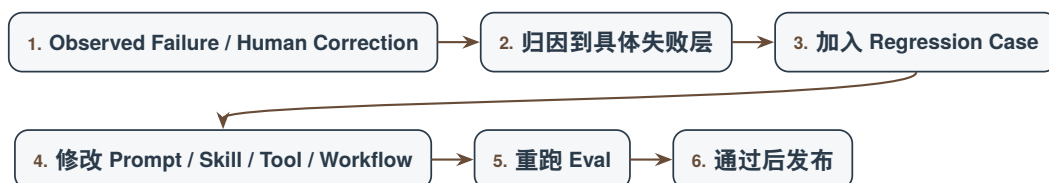
9.1 Defaults、Examples 与 Feedback：把领域经验变成可维护行为

Agent 的行为质量不能只靠让最终用户不断“写更好的 Prompt”。对高频、可重复的领域任务，系统设计者通常应该先提供经过验证的默认行为，再开放必要的配置面。这背后的原因不是用户不会写 Prompt，而是系统 Owner 才拥有完整的业务约束、风险边界、历史失败案例和评测集。

稳定的领域行为可以分成三层：

- **Defaults / Policy**：默认模型、工具权限、回答格式、必须检查的字段、停止条件和升级人工的规则；
- **Examples**：代表真实分布的正例、反例和边界例，用来说明“在这种输入下什么结果才算对”；
- **Feedback**：人工修正、失败 case、用户拒绝或业务结果，用来发现现有规则哪里失效。

三者的生命周期不能混在一起。一次人工修正只是 evidence，不应直接自动改写生产 Prompt、Skill 或 Long-term Memory。更稳妥的路径是：



这样“系统会学习”才有可审计含义：它不是看到一次反馈就永久改变行为，而是把反馈转换成可复现的测试证据，再决定是否升级 Canonical 行为。对于安全敏感、高价值业务尤其如此。

Examples 也不是越多越好。重复、互相矛盾或只覆盖 happy path 的示例会增加 Context 成本并制造

错误先验。选择示例时应覆盖关键分支和失败边界，并用真实 eval 证明加入这些 examples 后整体任务成功率提高，而不是只挑一个演示案例。

10 State 与 Memory：先区分“当前执行”与“以后还能记得”

一个独立的语言模型调用只根据这一次请求中可见的 Context 生成输出；应用跨轮次“记住”某件事，通常不是因为模型权重在每次聊天后被更新，而是因为运行时把信息持久化，并在后续调用中重新取回。某些托管 API 可以替应用保存 conversation/thread state，但这仍属于运行时状态管理，不等于模型参数获得了新的长期记忆。

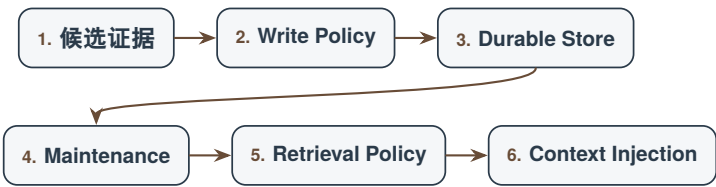
先把五个容易混淆的对象分开：

对象	真正保存什么
Conversation History	某段对话中的消息、模型输出与工具交互记录
Graph / Task State	当前 workflow 为继续执行而需要的结构化状态
Checkpoint	某个执行步骤的 State Snapshot，用于恢复、回放或人工中断
Long-term Memory	跨 thread / session 仍希望被后续任务重新调用的信息
Vector Store	一种可检索索引/存储方式；可以服务文档 RAG，也可以服务 Memory，但本身不等于 Memory

LangChain v1 Agent 的 short-term memory 以 Agent State + Checkpointer 为主；跨 thread 的持久信息可以放入 Store。LangGraph persistence 通过 checkpoint 支持 human-in-the-loop、fault recovery、replay / time-travel debugging 和 thread-level memory。开发阶段可以使用 InMemorySaver；生产环境需要数据库支持的持久化实现。Checkpoint 能恢复 workflow state，但不能自动解决业务系统里的重复写入和外部副作用。

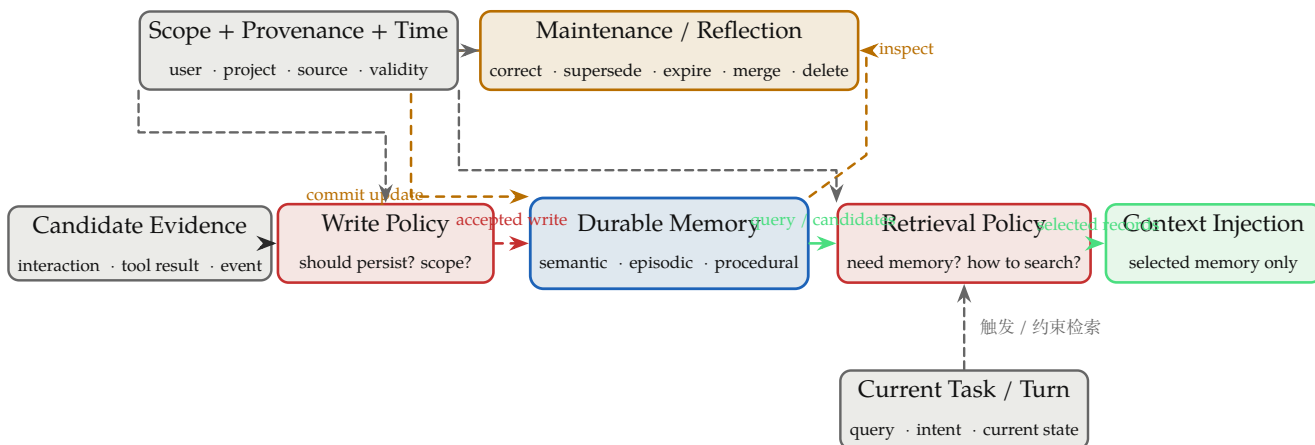
10.1 Long-term Memory 的母模型：写入、维护、检索、注入是四个决策点

把一切历史记录都保存下来，不等于建立了可用的记忆系统。一个真正的长期记忆闭环至少包含：



这里的箭头表示数据进入下一阶段的运行顺序；每一阶段都可以由确定性代码、模型判断或人工控制，并不要求全部交给同一个 Agent。

长期记忆不是一条“保存历史”的流水线：写入、维护、检索、注入分别有自己的触发条件与失败模式。



Reject / Skip 没有画成进入 Memory 或 Context 的箭头；未通过 gate 的信息就停在该决策点。

Figure 6 Agent 长期记忆的三个生命周期接口。左侧写入路径决定哪些证据可以进入 Durable Memory；右侧读取路由 Current Task 触发检索并只把选中的记录注入 Context；上方维护路径负责更新、失效、合并和删除。Scope、Provenance 与 Time 同时约束写入、维护和检索。

这张图最重要的结论是：

Memory ≠ Conversation Log ≠ Retrieval Index

Conversation log 可以是生成记忆的原始证据，vector / FTS / graph 可以是检索记忆的实现，而 Memory 是应用决定长期保留并在未来任务中重新使用的状态。

10.2 第一维：记什么——Semantic、Episodic、Procedural

Agent 工程里常借用三类记忆名称来组织不同内容。它们是实用的工程分类，不应被误写成对人类认知记忆系统的完整模型。

类型	主要保存什么	Agent 例子
Semantic Memory	相对稳定的事实、偏好、关系与当前已知状态	用户偏好暗色模式；某项目使用 PostgreSQL；Alex 负责 ML team
Episodic Memory	带时间和情境的过去经历、事件或成功/失败轨迹	上周一次部署失败的上下文与处理结果；某次对话中已经完成的动作
Procedural Memory	“应该怎样做”的规则、技能、提示与操作流程	系统指令、SKILL.md、固定 SOP、从成功轨迹提炼出的执行步骤

这三个类别回答的是“**记忆内容的语义是什么**”，不是“**用什么数据库存**”。同一个 Semantic Memory 可以放在 Markdown、SQLite、Postgres、vector index 或 graph 中；同一个数据库也可以同时存 semantic 与 episodic records。

10.3 第二维：怎么存——五种常见架构模式可以叠加

实际系统里常见下面五种模式。它们并不完全处在同一个抽象层：前三种更接近存储/检索表示，后两种把抽取、更新或关系建模提升成更完整的 memory layer。因此不要把它们理解成互斥的五选一。

模式	核心机制	适合与代价
Bounded Prompt / File Memory	少量高价值事实保存在 Markdown / profile，并在会话开始时直接注入 Context	最透明、易人工编辑；但受 Context/token 预算限制，过长后会增加干扰，且需要主动清理旧信息
Relational + Lexical Memory	SQLite/Postgres 行记录 + metadata/time + FTS/BM25 关键词检索	精确字段、日期、ID 和审计友好；对改写、同义词与跨语言表达的召回能力有限
Vector-backed Semantic Memory	文本/记录生成 embedding，使用 cosine / dot-product 等相似度召回，并结合 metadata filter	擅长语义近邻和改写；需要管理 embedding model、索引、阈值、误召回和内容更新后的向量同步
Managed Memory Layer	由 memory manager 从交互中抽取、合并、更新、删除或检索记忆，底层可再接普通 Store / Vector Store	少写业务胶水，但多了一层 LLM 判断、延迟、成本和错误写入风险；必须评估 extraction/update policy
Temporal Knowledge Graph	把 entity / relation / episode 与有效时间组织成图，再结合 lexical/semantic search 与 graph traversal	适合关系、多跳和随时间变化的事实；抽取、实体消歧、图维护和查询更复杂，小型偏好记忆可能明显过度设计

当前系统可以帮助看清这些模式怎样组合：

- **Hermes Agent** 当前把 `MEMORY.md` / `USER.md` 作为有界、人工可读的持久记忆，并在 `session start` 注入一个 `frozen snapshot`；完整历史另外保存在 `/.hermes/state.db`，通过 `SQLite FTS5` 做 `session search`。也就是说“`prompt memory`”和“可搜索历史”是两条不同路径。
- **Waku Agent** 当前默认用 `.waku/state.db` 保存 `facts` / `episodes`，并用 `FTS5` 做关键词检索，同时生成可读的 `MEMORY.md mirror`；它把 `semantic`、`episodic`、`procedural memory`、`retrieval gate` 与 `consolidation` 明确拆成模块，并允许把 `semantic store` 升级到 `Supabase pgvector`、`Mem0`、`Zep` 或 `LangMem` 等后端。
- **Supabase + pgvector** 仍然是 `Postgres`： `pgvector` 给普通表增加 `vector column`、`distance operator` 与 `ANN index`。它可以承载 `semantic memory`，但“关系数据库”和“向量检索”并不冲突。
- **LangMem** 更像 `memory-management toolkit`：它区分 `semantic` / `episodic` / `procedural memory`，支持 `hot-path` 与 `background formation`，并可以通过 `LangGraph BaseStore` 连接持久化 `Store`。它不是一种固定数据库形态。

10.4 第三维：怎么找——Preload、Lexical、Semantic、Graph/Hybrid

`Memory` 是否能保存，和“这一轮能不能找到它”是两个问题。常见 `retrieval path` 可以分为：

1. **Preload / Always-in-Context**：极小、极高价值的 `profile` / `instruction` 直接随每轮 `Context` 注入；没有搜索延迟，但会永久占用 `token`，并可能让无关旧记忆持续影响模型。
2. **Lexical Retrieval**： `SQLite FTS5`、`Postgres full-text`、`BM25` 等根据 `token`/词项匹配；适合名字、ID、错误码、日期和精确关键词。
3. **Semantic Retrieval**：将 `query` 与 `memory record` 映射为同一 `embedding space`，再做相似度检索；适合同义改写和语义近邻，但 `embedding` 相似并不等于事实相关或事实仍然有效。
4. **Graph / Hybrid Retrieval**：用 `entity`、`relation`、时间和 `graph distance` 扩展或 `rerank` 候选，并可与 `lexical` / `semantic retrieval` 组合。图数据库本身并不要求“所有节点和边都必须做 `embedding`”；向量只是其中一种检索信号。

因此一个很实用的设计是先加 **Retrieval Gate**：判断当前 `turn` 是否真的需要长期记忆，再决定是否支付搜索、`embedding`、`rerank` 和 `Context token` 成本。`Waku` 当前就把这个 `gate` 作为独立模块；但 `gate` 自身也有一次判断成本，而且 `false negative` 会导致“明明记得却没有取回”。

10.5 第四维：怎么维护——不要把“新事实”简单等同于覆盖旧行

长期运行以后，`Memory` 的主要难题通常从“存不下来”转成“旧事实、重复事实和冲突事实越来越多”。维护操作至少要区分：

- **No-op / Reject**: 这条信息没有长期价值，或来源不可信，不进入 memory；
- **Insert**: 新增独立事实或 episode；
- **Update / Correct**: 旧记录本身错误，需要改正；
- **Supersede / Invalidate**: 旧事实曾经正确，但现在不再有效；保留历史，并记录新的有效区间；
- **Expire / TTL**: 信息只在有限时间内有价值，到期后不再参与默认检索；
- **Delete**: 因为用户请求、隐私、合规、错误数据或保留策略真正删除；
- **Consolidate / Reflect**: 把多条重复、零碎记录合并成更稳定的事实、episode summary 或 procedural rule。

例如“产品发布日期原定 5 月 20 日，后来改为 6 月 3 日”不是简单的真假冲突。若未来要回答“5 月初我们当时计划什么时候发布”，旧值仍有历史意义。Temporal memory 因此需要至少区分：

recorded/observed time 与 fact validity time.

Zep / Graphiti 的 temporal graph 就会在 relation/fact 上维护类似 valid_at / invalid_at 的时间字段，使新信息可以 supersede 旧关系而不必直接抹去历史。它的当前检索也可以结合 semantic、keyword 和 graph-distance signals；这比“图数据库就是把节点和边都嵌入以后做向量搜索”更准确。

10.6 Hot Path 与 Background：记忆形成发生在什么时候

写 Memory 也有运行时选择：

- **Hot Path**: 在当前交互中立即判断并写入。关键偏好或用户明确说“记住这个”时很合适；缺点是增加响应延迟，而且当前对话尚未结束时可能过早总结。
- **Background / Reflection**: 对话结束或积累若干 turns 后再批量抽取、去重、合并与更新。不会阻塞前台回答，更容易获得完整上下文；代价是记忆有延迟，后台任务还要处理重试、重复执行和最终一致性。

LangMem 当前明确支持 hot-path memory tools 与 background ReflectionExecutor / memory manager；Waku 也把若干聊天后的 consolidation 与当前 turn retrieval 分开。所谓“睡眠/反思”最稳妥的工程含义就是：**把不需要阻塞用户的 memory maintenance 从主请求路径移到后台处理**，而不是假设 Agent 在离线时会自动获得更强的认知能力。

10.7 当前实现差异：产品名不能替代架构判断

版本变化很快，尤其不能把某个产品旧版的 memory policy 写成永久事实：

- **Mem0 OSS v3 (2026)** 当前已经改为 single-pass ADD-only extraction；检索融合 semantic、BM25 keyword 与 entity matching。旧版“同一抽取器在 ADD / UPDATE / DELETE 之间决策”的描述不再适合作为当前 OSS 默认行为。
- Mem0 的旧 OSS external graph-store integration 已移除；当前 Graph Memory 属于 Mem0 Platform 内建能力。因此“Mem0 OSS 默认同时维护外部 Neo4j 图”已经过时。
- **LangMem** 的 memory manager 仍可按配置执行 insert / update / delete，而且它把 memory type、memory formation 与底层 Store 分开；因此它和 Mem0 当前 OSS 的 write policy 不能混成同一个模型。
- **Zep / Graphiti** 的价值重点在 temporally-aware knowledge graph，而不是“图一定比向量更高级”。对于少量独立偏好，图抽取与异步 ingest 的复杂度可能没有收益；只有关系、多跳和时间演化确实重要时才值得承担这层成本。

单次 demo 中某个 backend 用了 4 秒、10 秒甚至更久，只能说明那个具体运行的 model、network、ingestion path 和数据规模；不能直接推成“某类 memory architecture 永远更慢”。要比较系统，至少固定 corpus、query set、LLM/embedding model、top-k、写入生命周期和评测方法，并报告多次运行的 latency distribution。

10.8 Memory 的安全边界：持久化错误比一次 hallucination 更危险

Memory 会在未来反复进入 Context，所以错误写入、恶意写入和跨用户泄漏具有**持久化放大效应**。生产系统至少要处理：

- **Scope isolation**：user / tenant / project / agent namespace 必须明确，不能让一个用户的 memory 被另一个用户检索到；
- **Provenance**：记录信息来自用户明确陈述、工具结果、业务数据库、网页还是模型推断，必要时保留 source id / timestamp；
- **Trust boundary**：网页、邮件、工具返回中的 prompt injection 不能未经审查直接升级为 procedural memory 或高优先级 instruction；
- **Sensitive data policy**：凭据、密钥、受限 PII 不应因为“以后可能有用”就进入普通 memory store；
- **Deletion / retention**：用户撤回、法规要求、项目结束和错误修正必须有真实删除/失效路径；
- **Auditability**：应能回答谁写了这条 memory、何时写、后来怎样被更新、哪一次回答检索并使用了它。

对于会直接进入 system prompt 的 curated memory，write gate 应比普通文档索引更严格，因为一条恶意或错误 instruction 可能跨很多 future turns 持续影响行为。

10.9 Memory 怎么评：不能只问“最终回答看起来对不对”

至少把评测拆成五层：

评测层	要回答的问题
Write Quality	应该记的是否被捕获；不该记的是否被拒绝；抽取事实是否忠实于来源
Maintenance Quality	更新、supersede、expire、delete、deduplicate 是否处理正确；旧事实会不会错误压过新事实
Retrieval Quality	需要时能否召回；top-k 中无关 memory 有多少；跨语言/改写/时间查询是否稳定
End-to-end Utility	注入 memory 后任务成功率是否提高；是否反而引入偏见、过度解释或 stale-memory error
System Cost / Safety	write/read latency、额外 tokens、embedding/LLM cost、跨租户泄漏、memory poisoning 与删除合规是否可接受

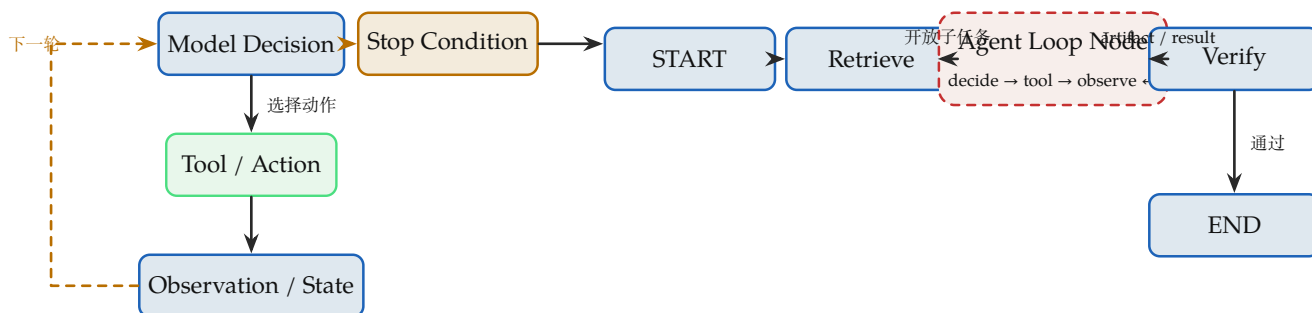
因此“记得更多”不是目标。更好的目标是：只把未来任务真正需要、来源可追溯、生命周期清楚的信息，在需要的时候准确取回，并以可控成本注入 Context。

11 Loop 与 Graph：开放探索和显式流程不是二选一

“Loop engineering”和“Graph engineering”容易被讲成两代互相替代的 Agent 技术，实际更稳定的区别是：下一步控制权在运行时由模型决定多少，又有多少已经被系统设计者显式写进控制流。

结构	控制方式	适合的问题
Agent Loop	Model 根据当前 State / Observation 反复选择下一动作，直到满足 stop condition	开放搜索、调研、调试、未知工具序列；下一步无法预先稳定确定
Workflow / Graph	主要阶段、节点、边、并行关系和部分分支在设计时显式定义	审批、退款、ETL、固定业务流程；需要可预测路径、恢复和审计
Hybrid	Graph 规定大结构，其中某些 Node 内运行 Agent Loop；Loop 也可调用一个固定 subworkflow	生产系统最常见：稳定业务骨架 + 少量开放问题求解

Graph 也不等于“全部确定性”。Node 可以是普通函数、LLM call、Tool、router 或完整 Agent；Edge 也可以根据运行时 state 条件选择。Graph 真正提供的是显式拓扑和状态转移边界，不是消灭模型随机性。

Agent Loop: 下一步由运行时观察决定**Graph / Workflow: 大结构显式, 局部仍可动态**

左图的循环边本身就是主控制结构；右图的 START / Retrieve / Agent Loop / Verify / END 已预先固定，只有 Agent Loop Node 内部保留开放决策。因此 Graph 与 Loop 的差异是控制权落在哪里，不是“确定性系统”和“智能系统”的二选一。

Figure 7 Loop 与 Graph 的控制权差异。左侧 Loop 的下一动作在运行时动态决定；右侧 Graph 先固定主要节点与边，但其中一个 Node 仍然可以包含完整 Agent Loop。箭头表示控制流，不表示理论上的技术演进顺序。

因此第一判断不是“Loop 还是 Graph 哪个先进”，而是：

1. 哪些步骤的顺序和合法分支已经由业务规则确定？先写进 Workflow / Graph；
2. 哪些子问题必须根据未知 Observation 临场选择下一动作？把局部控制权交给 Agent Loop；
3. 哪些写操作必须经过审批、幂等和固定补偿路径？不要让自由 Loop 拥有无限制副作用；
4. 每个 Loop 的 stop condition、iteration / time / cost budget 是什么？

这也是为什么“固定退货规则”通常不需要 Planner Agent，而 Deep Research、未知 bug 调试或开放网页调查更适合保留 Loop。

12 Multi-Agent: 只有任务真的可分解时，并行委派才有价值

Subagent 只是一次隔离委派；Multi-Agent 则要求多个 Agent runs 共同完成同一最终任务。最常见的稳定结构是 orchestrator-worker：



其中 Orchestrator Own 的不是“把所有细节再做一遍”，而是 decomposition、worker boundary、budget、progress、gap detection 与 synthesis。Worker 应收到明确的 objective、scope、允许的 tools、期望输出格式和 stop condition；否则多个 worker 很容易重复同一搜索、遗漏边界或产生互相矛盾的结论。

12.1 什么时候值得使用多个 Agent

Multi-Agent 的主要收益来自可并行的独立子问题与 context isolation，而不是“多几个模型投票就一定更聪明”。优先考虑它的场景包括：

- 一个问题天然分成多个相对独立的研究方向，可以并行搜索；
- 每个分支会产生大量中间 Tool outputs，不希望污染主 Agent Context；
- 不同分支需要不同 tools、instructions 或 models；
- 单个 context window 无法经济地容纳全部探索过程；
- wall-clock latency 可以通过真实并行显著降低，而且任务价值足以覆盖额外成本。

反过来，如果任务强依赖共享中间状态、步骤高度串行、子问题之间不断互相修改，或者本来一次 Tool Call 就能完成，多 Agent 只会增加 token、同步、权限和失败面。

总成本至少会变成：

$$C_{\text{multi}} = C_{\text{orchestrator}} + \sum_{i=1}^k C_{\text{worker},i} + C_{\text{synthesis}} + C_{\text{coordination}}.$$

Anthropic 在其 2025 Research 系统的内部数据中报告：多 Agent Research 相比普通 chat 使用了约 15× tokens，并在特定 internal research eval 上获得明显收益；这只能作为“并行研究可能用成本换覆盖”的工程证据，不能推广成多 Agent 对任意任务都更好。

12.2 Deep Research：一个可复用的 Integration 母例

Deep Research 最值得保留的不是某个产品的 Agent 名称，而是这条可生成的结构：

Clarify / Scope → Research Brief → Orchestrate → Parallel Evidence Gathering → Synthesis → Citation / Verification

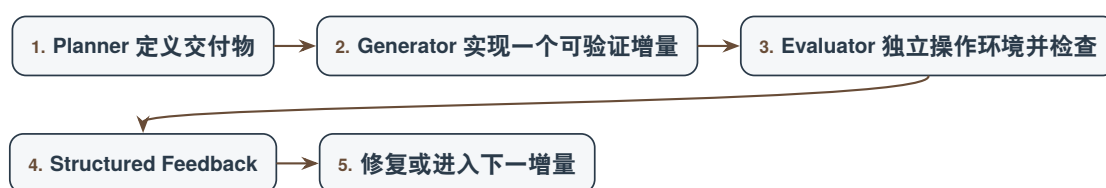
第一步先把问题边界和交付物写清；随后 Orchestrator 根据 brief 动态拆分研究方向；Workers 各自在隔离 Context 中搜索、读取和分析；完成后返回可验证 findings、sources 或 artifact references，而不是要求系统永久保存或暴露每个 Agent 的私有思维过程。最后由 synthesizer 解决重复、冲突、缺口和引用，再决定是否需要追加一轮研究。

这套结构同时展示了 Loop 与 Graph 的混合：整体生命周期可以显式编排，但“研究哪个来源、发现新线索后往哪里继续”仍属于 Worker Loop 的动态控制。

Multi-Agent 的正确性也必须分层验证：worker 是否完成自己边界内的任务；handoff 是否丢信息；synthesizer 是否忠实引用 worker artifacts；最终输出是否满足全局目标。只评最后一段文字，会看不见协调失败发生在哪一层。

12.3 Planner-Generator-Evaluator：多 Agent 的另一条价值是分离“做”和“判”

Multi-Agent 不只用于把搜索并行化。另一个高价值结构是把生成工作与评价工作放进不同 execution contexts：



这一结构针对的是 self-evaluation bias：同一个 Agent 在刚生成结果后再问自己“做得好吗”，很容易沿用已有解释并过度宽容。把 evaluator 隔离出来以后，可以给它独立 rubric、Tools 和失败阈值，让它从环境结果而不是生成者叙述出发判断。

对于软件工程，最强的 evaluator 往往不是“再读一遍代码”，而是运行 tests、浏览真实 UI、调用 API、检查数据库状态和重现用户路径。对于主观任务，也应尽量把“好不好”拆成可操作 criterion，例如 originality、craft、functionality，而不是只让另一个 LLM 给总分。

Anthropic 2026 的长时程应用实验进一步使用了 **sprint contract**：Generator 和 Evaluator 在写代码前先明确这一小段工作“完成”的可测试定义，再进入实现。它可以抽象成：

Goal → Acceptance Contract → Implementation → Independent Verification

这比“让多个 Agent 互相讨论直到满意”为何更稳：每一轮都有外部可检查的完成条件，feedback 能直接转成下一次修改，而不是继续扩散自然语言意见。

这种结构同样不是免费收益。独立 evaluator 会显著增加模型调用、工具执行和 wall-clock time；只有任务价值、失败成本或质量要求足以覆盖验证成本时才值得使用。

13 LangChain 与 LangGraph：当前分工

13.1 LangChain v1：高层 Agent API

当前 LangChain 的标准 Agent API 是 `langchain.agents.create_agent`：

```

1 from langchain.agents import create_agent
2
3 agent = create_agent(
4     model=model,
5     tools=[search_docs, lookup_order],
6     system_prompt="Use tools only when needed and cite evidence.",
7 )

```

`create_agent` 运行在 LangGraph runtime 上。LangChain 更适合快速组合标准 tool-calling agent、provider abstraction、middleware 与 structured output。

“代码少了”不代表系统复杂度消失。权限、状态、失败处理、评测和观测仍然由应用负责。

13.2 Deep Agents：当前一个明确的 Harness 实现

LangChain 当前把 `deepagents` 明确描述为 agent harness：底层仍是 tool-calling loop，但默认组合 planning、filesystem context management、subagent delegation、long-term memory、permission rules 与 human-in-the-loop 等能力，并运行在 LangGraph runtime 上。

这正好说明 Framework、Runtime 与 Harness 可以分层：LangGraph 提供 durable execution / state orchestration；LangChain 提供较高层 Agent API；Deep Agents 再提供更 opinionated 的 long-running Agent Harness。简单任务仍应优先使用更薄的 `create_agent` 或普通 LangGraph workflow，而不是默认加载全部 Harness 组件。

13.3 LangGraph v1：显式 State + Node + Edge

LangGraph 适合需要 multi-step workflow、conditional branch、loop、parallel branch、human approval、interrupt / resume、durable execution 和显式 state machine 的场景。当前官方文档也明确区分：workflow 的主要 code path 是预先定义的，而 agent 会在运行时动态决定 process / tool usage；两者都可以在同一个 LangGraph runtime 中组合。

核心模型可以写成：

$$\text{Node} : \text{State}_{in} \mapsto \Delta \text{State}$$

最小概念代码：

```

1 from typing_extensions import TypedDict
2 from langgraph.graph import StateGraph, START, END
3
4 class State(TypedDict):
5     query: str
6     docs: list[str]

```

```
7
8 def retrieve(state: State):
9     docs = search_index(state["query"])
10    return {"docs": docs}
11
12 builder = StateGraph(State)
13 builder.add_node("retrieve", retrieve)
14 builder.add_edge(START, "retrieve")
15 builder.add_edge("retrieve", END)
16 graph = builder.compile()
```

标准分工可以压成：

- High-level standard agent: LangChain create_agent;
- Custom stateful orchestration: LangGraph StateGraph。

如果任务只是确定性的 “Input → Function → Output”，普通代码通常更简单，也更容易测试。

14 MCP：标准化外部 Context 与 Capability 接口

MCP (Model Context Protocol) 解决的是：不同 LLM Host 怎样通过标准协议发现并调用外部 Context / Capability，而不是为每一个应用和每一个外部系统重新写一套私有集成。

最小角色是：

- **Host**：运行 LLM 应用的宿主；
- **Client**：Host 内负责某条 MCP 连接的组件；
- **Server**：暴露 Context / Capability 的服务。

Server 的核心 primitives 仍包括 Tools、Resources 与 Prompts。

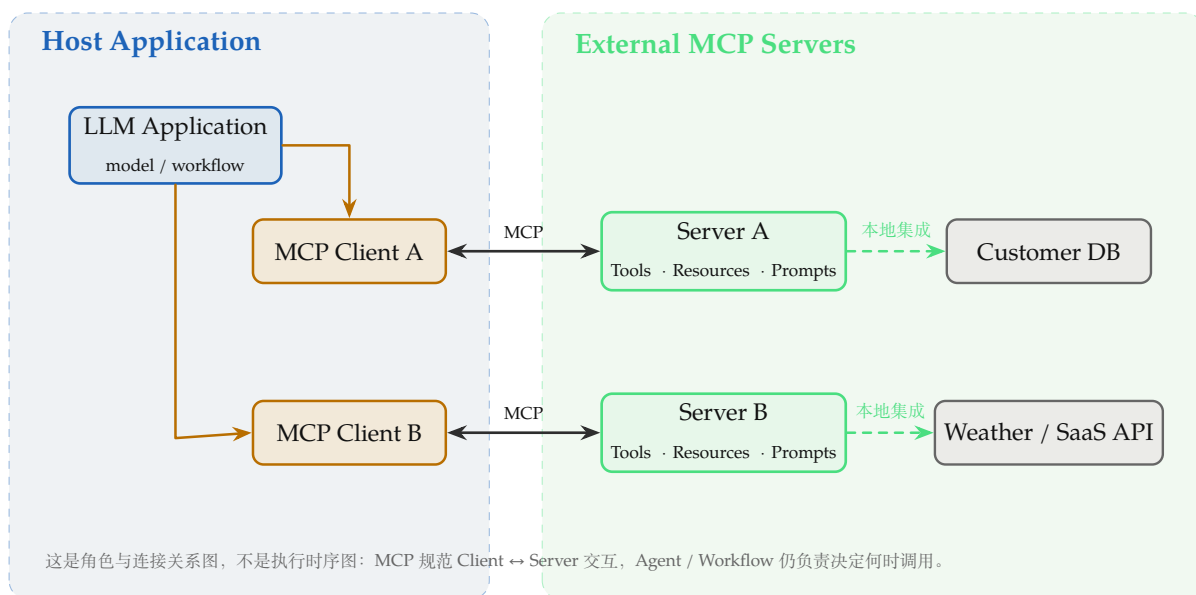


Figure 8 MCP 的静态角色边界。Host 内的 Client 与外部 Server 通过 MCP 协议交互；Server 再把自己的 Tool、Resource 或 Prompt 暴露给 Host。图中连接表示协议交互，不表示 Agent 的任务决策顺序。

14.1 当前 MCP 基线：2026-07-28

截至 2026-08-22，当前 MCP 规范版本是 2026-07-28。新版核心采用 stateless request/response 模型：现代客户端通过 server/discover 获取服务器身份与能力，不再要求每个连接先建立 protocol-level session；Streamable HTTP 的现代路径也不再依赖 Mcp-Session-Id 绑定 worker。

新版还引入或强化了：

- Multi Round-Trip Requests；
- header-based routing；
- cacheable list results；
- authorization hardening；
- Extensions framework；
- formal deprecation policy。

Roots、Sampling 与 MCP-level Logging 已进入 deprecated 路径；ping 被移出新协议。兼容旧 server 时，当前 Python SDK v2 的 Client 可以回退到 2025-era initialize handshake。

14.2 MCP Python SDK v2

当前官方 Python SDK 的稳定线是 v2，要求 Python 3.10+。高层 Server 类已经是 MCP Server：


```
1 from mcp.server import MCPServer
2
3 mcp = MCPServer("Demo")
4
5 @mcp.tool()
6 def add(a: int, b: int) -> int:
7     """Add two integers."""
8     return a + b
9
10 if __name__ == "__main__":
11     mcp.run() # default transport: stdio
```

SDK 可以从函数名、docstring 与 type hints 推导 Tool 的名称、说明和 input schema。`@mcp.tool()` 的括号属于当前 API 契约。

当前主要 transport 包括：

- stdio：本地 child process；
- Streamable HTTP：remote server；
- SSE：仍存在兼容支持，但已经是旧一代路径，不应作为新系统首选。

stdio 模式下 stdout 是协议通道，业务日志应走 stderr / logging，避免污染 wire protocol。

14.3 Tool Schema 与错误边界

Tool 至少需要结构化 input schema。当前 Python SDK 可以从 type hints 自动生成 JSON Schema。

可由模型调整参数后重试的业务错误，应该返回可操作的 execution feedback；协议结构本身错误则属于更低层的 RPC / protocol failure。两类错误如果混在一起，模型和 Runtime 都很难知道该重试、改参数还是直接终止。

15 Security：任何 RAG / Tool / MCP 系统都要画 Trust Boundary

真实 Agent 会同时接收用户输入、检索内容和 Tool / MCP 结果。这三类输入都可能不可信。

主要攻击面包括：

- Prompt Injection；
- Retrieval Poisoning；
- Tool Injection；
- Data Exfiltration；

- Over-permissioned Tool;
- Secret / PII Leakage;
- Arbitrary Code Execution;
- Malicious MCP Server / Tool Result。

15.1 先限制 Blast Radius，再讨论模型会不会犯错

Agent 风险可以用一个非常粗但有用的分解来思考：

$$Risk \approx P(\text{unsafe action}) \times \text{BlastRadius}$$

Prompt、Guardrail、模型训练与人工审批主要尝试降低前一项；Sandbox、filesystem boundary、network egress policy、least privilege、credential isolation 与 transaction boundary 主要限制后一项。因为任何概率型检测都可能漏掉，生产系统不能把“模型通常会拒绝”当成唯一安全边界。

这也解释了为什么高自主 Agent 越来越强调 **environment-level containment**：

- 不让不可信代码所在 Sandbox 看见长期 credentials；
- 文件系统只挂载当前任务真正需要的目录；
- 网络访问使用 allowlist / proxy / egress policy；
- 高风险 Tool 使用独立授权域和 side-effect ledger；
- Tool Result 仍按不可信外部输入检查，因为可信 Tool 也可能返回被注入的网页、README 或用户数据。

Human approval 很重要，但不应成为唯一防线。高频 permission prompt 会产生 approval fatigue；更稳的结构是把低风险动作放进受限环境自动执行，把真正高影响且用户能判断的少数决策保留给人工确认。

Trust Boundary | 进入 Context 的文本不自动拥有指令权限

外部文档、网页、数据库字段和 Tool result 首先都是数据。系统必须把 Trusted Policy / Instructions 与 Untrusted Data / Evidence 分开，不能因为一段文本进入 Context 就默认它可以覆盖上层指令。

生产 Tool 至少需要：input validation、authentication、authorization、least privilege、timeout、retry policy、rate limit、audit log、side-effect classification、idempotency strategy、rollback / compensation 与必要的 user confirmation。

search_docs() 这样的只读查询，与 send_email()、charge_card()、delete_file()、update_database() 这样的有副作用动作，不能共享同一授权和重试策略。

对于 HTTP-based MCP server，授权应遵循 MCP 当前 authorization contract；studio transport 通常从运行环境获得 credentials，不走相同 HTTP OAuth 流程。无论使用哪种 transport，都必须把权限缩到完成任务所需的最小范围。

16 Observability、Evaluation、Cost 与 Failure Recovery

16.1 Observability：必须能重建一次执行轨迹

每次 Agent / Workflow 执行至少应该能够追踪：

- User Request；
- Context Snapshot / References；
- Retrieved Chunks；
- Model Calls；
- Tool Calls / Arguments / Results；
- State Transitions；
- Retries / Errors；
- Token Usage；
- Latency / Cost；
- Final Output。

否则只能看到最后答案，无法判断错误发生在 Retrieval、Model、Tool、State 还是 Routing。

16.2 Evaluation：按层拆开，而不是只看“回答准确率”

层	典型检查对象
Retrieval	Recall@k、precision / relevance、MRR / ranking quality、permission-filter correctness
Generation	answer correctness、groundedness、citation correctness、refusal correctness、structured-output validity
Agent	task success、tool selection、tool argument accuracy、trajectory quality、unnecessary calls、loop / stop correctness、safety violations
System	P50 / P95 latency、token / tool cost、failure rate、recovery success、framework / model update regression

任何 Model、Prompt、Retriever、Framework 或 MCP Server 更新，都应能跑 regression eval，而不是只靠人工试几次。

16.3 Agent Eval 的基本单位：Task、Trial、Grader、Transcript

Agent Eval 和单轮问答最大的区别是：结果会受到多步决策、工具、环境状态和随机性的共同影响。为了避免“跑一次看起来不错”这种假精确，至少把四个对象分开：

对象	含义
Task	一个有明确输入、环境初始状态和 success criteria 的测试问题
Trial	同一个 Task 的一次独立运行；因为 Agent path 有随机性，一个 Task 通常需要多个 Trials
Grader	对 outcome 或 trajectory 做检查的代码、模型或人工规则；一个 Task 可以有多个 Graders
Transcript / Trace	一次 Trial 的完整执行证据，包括模型 turns、tool calls、observations、state changes 与 final outcome

因此一个有意义的结果通常是分布而不是单点：

$$\text{Task Success Rate} = \frac{\text{successful trials}}{\text{all comparable trials}}.$$

如果只保存 final answer，就无法判断失败是路径偶然、Tool 环境异常，还是 Harness / Model 的稳定缺陷。

16.4 Agentic Benchmark 里 Runtime 本身就是实验变量

传统静态 benchmark 常把模型输出直接交给评分器；Agentic benchmark 则让模型在完整环境里安装依赖、运行测试、读写文件并迭代。因此 CPU、RAM、timeout、network、container policy、package cache 和并发策略都会改变实际可解问题集合。

Anthropic 2026 对 Terminal-Bench 2.0 的实验中，仅基础设施资源配置就造成最高约 6 个百分点的 score spread。这一数字只属于该实验，不能推广成固定阈值；但它证明了一个更一般的实验原则：

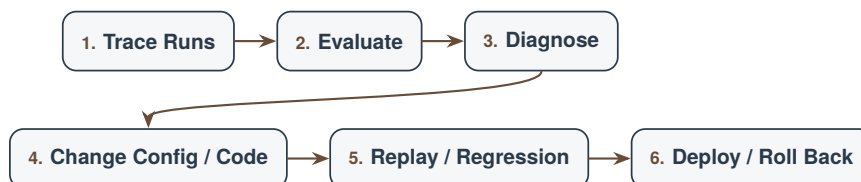
Agent Eval 可比性

比较两个模型或两个 Harness 前，必须同时固定并报告 Model Config、Prompt/Harness、Tool Versions、Environment Image、CPU/RAM、Timeout、Network Policy、Retry Policy 与 Trial Count。否则小幅 leaderboard 差异可能只是 Runtime 条件不同。

这也是为什么 Agent Eval 应同时保存 outcome、trajectory 与 environment metadata。

16.5 LLMOps: Trace 只有进入评测与回放闭环才会产生改进

“记录很多日志”不等于 LLMOps。一个可维护的 Agent 系统至少需要形成：



其中：

- **Trace** 保存足以重建行为的可观察事实：inputs/references、model/tool calls、tool results、state transitions、errors、latency、tokens、cost；不要求记录模型不可用的私有 chain-of-thought；
- **Evaluate** 使用 deterministic assertions、reference-based metrics、task-specific graders、必要时 LLM-as-judge 与 human review；
- **Diagnose** 必须定位 first failing layer，是 retrieval miss、bad routing、tool schema、permission、stop condition、model response 还是业务 side effect；
- **Change** 把 prompt、model、retrieval config、tool definition、memory policy 与 workflow topology 都作为可版本化配置，而不是线上随手改字符串；
- **Replay / Regression** 在固定 eval set 与历史失败 case 上重跑，证明新版本修复目标问题且没有制造新的回归；
- **Deploy** 对高风险系统保留 staged rollout、canary、rollback 与变更审计。

自动“自我改进”尤其需要这一层约束。Agent 可以提出新的 prompt、skill 或 memory rule，但不能因为一次运行看起来成功就直接把它提升为生产默认；先经过可重复 eval 与 review，才能把一次经验变成稳定系统行为。

Trace 本身也属于敏感数据资产。用户输入、Tool arguments/results、Memory、身份信息和 secrets 进入 tracing backend 前应执行最小化、redaction、retention 与 access-control policy。

16.6 Cost 与 Latency 是多步系统的累积量

一次 Agent 任务的总成本可以近似写成：

$$C_{\text{total}} = \sum_i C_{\text{model},i} + \sum_j C_{\text{tool},j} + C_{\text{retrieval}} + C_{\text{infra}}.$$

总延迟可以近似写成：

$$T_{\text{total}} \approx T_{\text{retrieval}} + \sum_i T_{\text{model},i} + \sum_j T_{\text{tool},j} + T_{\text{network}}.$$

更多 Context、更多 Agent iterations、更多 Tools 和更多 verification passes 都可能提高质量，也都会提高 cost / latency。是否值得只能通过实际任务评测决定。

16.7 Failure Recovery: 失败是运行时的一部分

至少要设计：

- LLM / Retriever timeout;
- MCP server unavailable;
- Tool validation / business error;
- partial write;
- duplicate retry;
- process crash;
- graph loop;
- stale checkpoint;
- malformed structured output。

读操作重试通常比较安全；写操作必须考虑 idempotency；破坏性写入通常需要 explicit confirmation 或 compensation。LangGraph checkpoint 可以恢复 workflow state，但不能自动替业务系统解决重复副作用。

17 Complexity Ladder：什么时候不要继续增加 Agent 自主性

层级	结构	什么时候足够
Level 0	普通确定性代码	规则已知，输入输出关系可以直接写成程序
Level 1	Single LLM Call	summarization、extraction、classification、one-shot generation / reasoning
Level 2	Retrieval / RAG	需要私有知识、大量文档、最新资料或 citation
Level 3	Tool Calling	模型需要读取或操作实时外部系统
Level 4	Fixed Workflow / Graph	有固定阶段、分支、retry、approval、checkpoint 和明确 state transition
Level 5	Autonomous Agent	下一步无法在设计时可靠确定，需要模型根据环境与 Tool observation 动态选动作

MCP 不是 Level 6。它横切 Level 3–5，为外部 Context / Capability 提供标准化连接方式。Multi-Agent 也不是 Level 6；它是对某些可分解任务进行 context isolation、specialization 或 parallel scaling 的组织方式，可以出现在 Workflow 或 Autonomous Agent 内部。

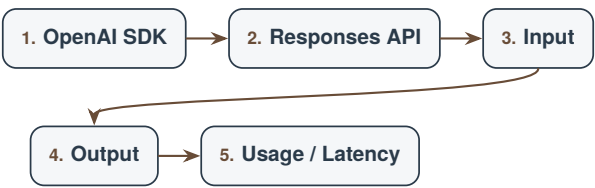
复杂度升级判据

能用普通函数解决，就不要上 Agent；能用固定 Workflow 清楚表达控制流，就不要把每一步都交给模型自由决定。只有“下一步本身依赖开放环境和中间观察，设计时无法稳定穷举”时，更高自主性才有充分理由。

18 一条可执行的实验路线

18.1 Lab A | 直接 Model API

目标：理解最小 model runtime。



至少记录 model、input tokens、output tokens、latency 与 cost。

18.2 Lab B | Semantic Retrieval

建立: Documents → Chunk → Embedding → Index → Search。至少比较 keyword vs semantic、small vs large chunk、with vs without metadata filter。准备一个小型 labeled query set，不用单个 query 宣布效果。

18.3 Lab C | RAG

流程: Query → Retrieve Top-k → Build Context → Generate → Cite Sources。分别检查 retrieval hit、answer correctness、citation support，以及 evidence 不足时是否正确拒答。

18.4 Lab D | Function / Tool Calling

实现一个 read-only lookup 与一个 state-changing operation，对比 schema、permission、confirmation、retry 和 audit log。这样才能真正看到 Tool Safety 的差异。

18.5 Lab E | LangChain Agent

使用 `langchain.agents.create_agent` 组合 model + tools，重点记录 model decision → tool call → observation → next decision → stop 的完整 trajectory。

18.6 Lab F | LangGraph Workflow

把同一任务改成有显式 State 的图: START → Retrieve → Analyze → Need More Evidence?; 证据不足时回到 Retrieval / Tool，足够时进入 Generate → Verify → END。加入 checkpointer、iteration limit、interrupt / approval、deliberate failure 与 resume test。

18.7 Lab G | MCP Server

使用当前 Python SDK v2: `MCPServer` → `@mcp.tool()` → `stdio`，再改成 remote Streamable HTTP。至少暴露一个 Tool、一个 Resource、一个 Prompt，避免把 MCP 误解成只有 Tool API。

18.8 Lab H | Production Evaluation

最后主动注入 Normal Case、Edge Case、Prompt Injection、Retriever Miss、Tool Timeout、Tool Error、Duplicate Retry、Permission Denial、MCP Server Offline。系统只在 happy path 演示成功，还不能称为可靠。

18.9 Lab I | Long-term Memory Lifecycle

建立一组跨 session 的小型记忆测试集，而不是只问一个“还记得我吗”。至少包含：稳定偏好、精确 ID、跨语言改写、时间更新、旧事实 supersede、应该忘记的敏感信息、两个用户之间的隔离，以及一条来自不可信网页/工具输出的恶意 instruction。分别记录：write decision、stored representation、retrieved top-k、final answer、latency、token/cost 与后续删除结果。再比较 always-in-context、FTS/BM25、vector/hybrid 与 temporal graph 中真正适合当前数据结构的方案。

18.10 Lab J | Skill、Tool 与 Harness Boundary

选择一个重复但多步骤的任务，例如“初始化一个 RAG backend 并验证 health endpoint”。先只给 Agent 原始 shell/file Tools，再把稳定步骤写成 Skill。比较两次运行的 instruction repetition、错误率与 token usage。随后故意删除一个底层 Tool，验证 Skill 是否会因为“写了步骤”就 magically 获得执行能力；正确结果应该是 Skill 仍然只能指导，真正动作必须由 Tool/Runtime 提供。

18.11 Lab K | Loop、Graph 与 Hybrid

用同一个业务目标实现三版：纯 Agent Loop、固定 Graph、Graph 中嵌入一个 Agent research/debug loop。注入一个未知外部问题和一个确定性审批步骤，观察哪部分真正需要模型临场决策。记录 stop correctness、unnecessary tool calls、side-effect safety、latency 与 recovery complexity，避免只比较最终答案。

18.12 Lab L | Orchestrator-Worker Deep Research

准备一个可自然拆成 3-5 个独立方向的研究任务。Single-Agent 作为 baseline；Multi-Agent 版本由 Orchestrator 明确 worker objective、tools、budget、output schema，Workers 把 findings + sources 写入 artifacts，再由 Synthesizer 汇总。比较 coverage、duplicate work、handoff loss、wall-clock latency、total tokens/cost 与 citation correctness。如果任务本来高度串行，应看到多 Agent 的协调成本可能大于收益。

18.13 Lab M | Session、Context 与 Crash Recovery

构造一个会产生多个 artifacts 的长任务。把完整 event log / task state 存在 Context Window 外部；运行若干步骤后主动杀死 Harness 进程或丢弃当前 Context，再从 durable Session 恢复。检查：是否能定位最后一个已提交副作用、是否会重复写入、是否能重新选择必要历史而不是把全部 log 再塞回 Context。随后对比“只有 conversation summary、没有原始 event log”的版本，观察哪些错误变得不可审计或不可恢复。

18.14 Lab N | Agent Eval 的重复试验与 Runtime 控制

为同一 Task 至少运行多次 independent trials，并同时记录 outcome grader、trajectory grader、token/cost 和 environment metadata。再只改变一个 Runtime 变量，例如 timeout 或 RAM limit，观察 success distribution 是否变化。这个实验的目标不是追求某个 leaderboard 数字，而是亲自验证：Agentic Eval 测量的是 **Model + Harness + Tools + Environment** 的组合行为。

19 最终压缩：十四个问题复原整套系统

遇到任何 LLM / Agent 系统，依次问：

1. 模型当前真正看见什么？——Context；
2. 当前证据从哪里来？——Retrieval / Files / DB / API；
3. 证据怎样进入模型？——Context Construction / RAG；
4. 哪些信息要跨会话保留，何时写入、更新和取回？——Long-term Memory；
5. 模型只能回答，还是可以行动？——Tool Calling；
6. 重复任务的“怎样做”是否需要独立沉淀？——Skill / Procedural Knowledge；
7. 谁把 Context、Tools、Loop、State、Memory、Permissions 与 Trace 组装成可运行代理？——Agent Harness；
8. Session 与这一轮 Context 是否已经分开？——Durable Event Log / Context View；
9. 下一步是开放探索还是已知流程？——Loop / Graph / Hybrid；
10. 是否存在真正独立、值得隔离或并行的子任务？——Subagent / Multi-Agent；
11. 当前运行状态在哪里保存，失败后怎样恢复？——State / Checkpoint / Artifacts；
12. 真正执行副作用的环境与模型是否隔离？——Sandbox / Permission / Credential / Egress Boundary；
13. 外部能力怎样标准化接入？——MCP / Tool Contract；
14. 怎样证明修改后的系统真的更可靠？——Repeated Trials / Trace / Evaluation / Security / LLMOps Regression。

一页压缩

Context 决定模型这一轮能看见什么，而 Durable Session 保存 Context Window 外部可恢复的任务历史；Retrieval / RAG 决定外部证据怎样进入；Long-term Memory 负责跨任务信息的持久化与再调用；Skills 沉淀重复流程，Tools 提供实际动作；Agent Harness 把这些能力连同状态、权限、循环和

观测组织成可运行系统，并把 Model/Decision 与 Sandbox/Side Effects 解耦；Loop 处理下一步未知的开放任务，Graph 固定已知业务骨架，Multi-Agent 只在任务可分解或需要独立 evaluator 时扩展执行上下文；MCP 标准化外部能力接口；Containment 限制 Blast Radius, Repeated-trial Evaluation 与 LLMOps Regression 决定系统能否长期可靠演进。

20 当前技术快照与官方资料

本节只锁定快速变化的 API / protocol 行为，机制主干不依赖具体厂商版本。当前快照日期为 2026-08-22。

- OpenAI Models / GPT-5.6: <https://developers.openai.com/api/docs/models>
- OpenAI GPT-5.6 Sol: <https://developers.openai.com/api/docs/models/gpt-5.6-sol>
- OpenAI Responses API: <https://developers.openai.com/api/reference/resources/responses>
- OpenAI Vector Stores: <https://platform.openai.com/docs/api-reference/vector-stores>
- OpenAI Agents SDK: <https://openai.github.io/openai-agents-python/>
- OpenAI Agents SDK Tracing / Guardrails: <https://openai.github.io/openai-agents-python/tracing/>, <https://openai.github.io/openai-agents-python/guardrails/>
- LangChain Agents: <https://docs.langchain.com/oss/python/langchain/agents>
- LangGraph Workflows / Agents: <https://docs.langchain.com/oss/python/langgraph/workflows-agents>
- LangGraph Persistence: <https://docs.langchain.com/oss/python/langgraph/persistence>
- LangChain Deep Agents / Agent Harness: <https://docs.langchain.com/oss/python/deepagents/overview>
- Anthropic Agent Skills: <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
- Anthropic Multi-Agent Research: <https://www.anthropic.com/engineering/multi-agent-research-system>

- Anthropic Long-running Agent Harness: <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- Anthropic Harness Design for Long-running Apps: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Anthropic Managed Agents / Brain-Hands-Session: <https://www.anthropic.com/engineering/managed-agents>
- Anthropic Agent Containment: <https://www.anthropic.com/engineering/how-we-contain-claude>
- Anthropic Agent Evals / Infrastructure Noise: <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>, <https://www.anthropic.com/engineering/infrastructure-noise>
- MCP 2026-07-28: <https://blog.modelcontextprotocol.io/posts/2026-07-28/>
- MCP Python SDK v2: <https://github.com/modelcontextprotocol/python-sdk>
- pgvector: <https://github.com/pgvector/pgvector>
- Supabase Vector Columns / pgvector: <https://supabase.com/docs/guides/ai/vector-columns>
- Hermes Agent Persistent Memory: <https://github.com/NousResearch/hermes-agent>
- Waku Agent Memory Architecture: <https://github.com/ShenSeanChen/waku-agent>
- LangMem: <https://langchain-ai.github.io/langmem/>
- Mem0 OSS v3 Migration / Graph Memory: <https://github.com/mem0ai/mem0>
- Graphiti / Zep Temporal Knowledge Graph: <https://github.com/getzep/graphiti>